1D DP - Lecture 3

06 November 2025 13:46

* **Always Remember:→** Try Your Best to come up with the Recursive logic only, Because after that DP is just 5 minutes of work. – vHizen

* **Tribonacci Sequence →** 0, 1, 1, 2, 4, 7, 13

  Any number is sum of previous 3 values

Ques → find n^{th} Tribonacci

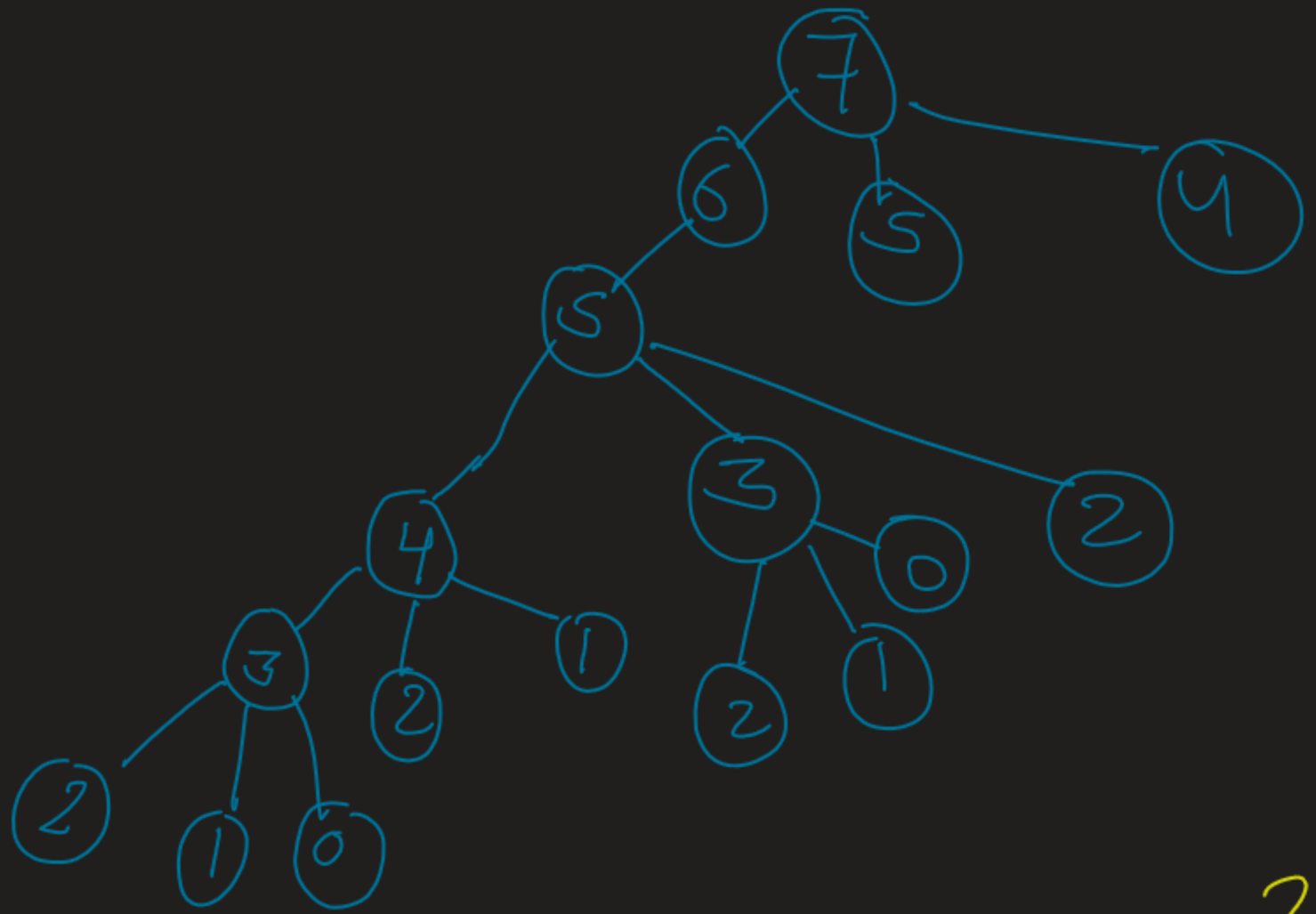
ex:– 3^{th} Trib. number → 2

5^{th} –|– → 7

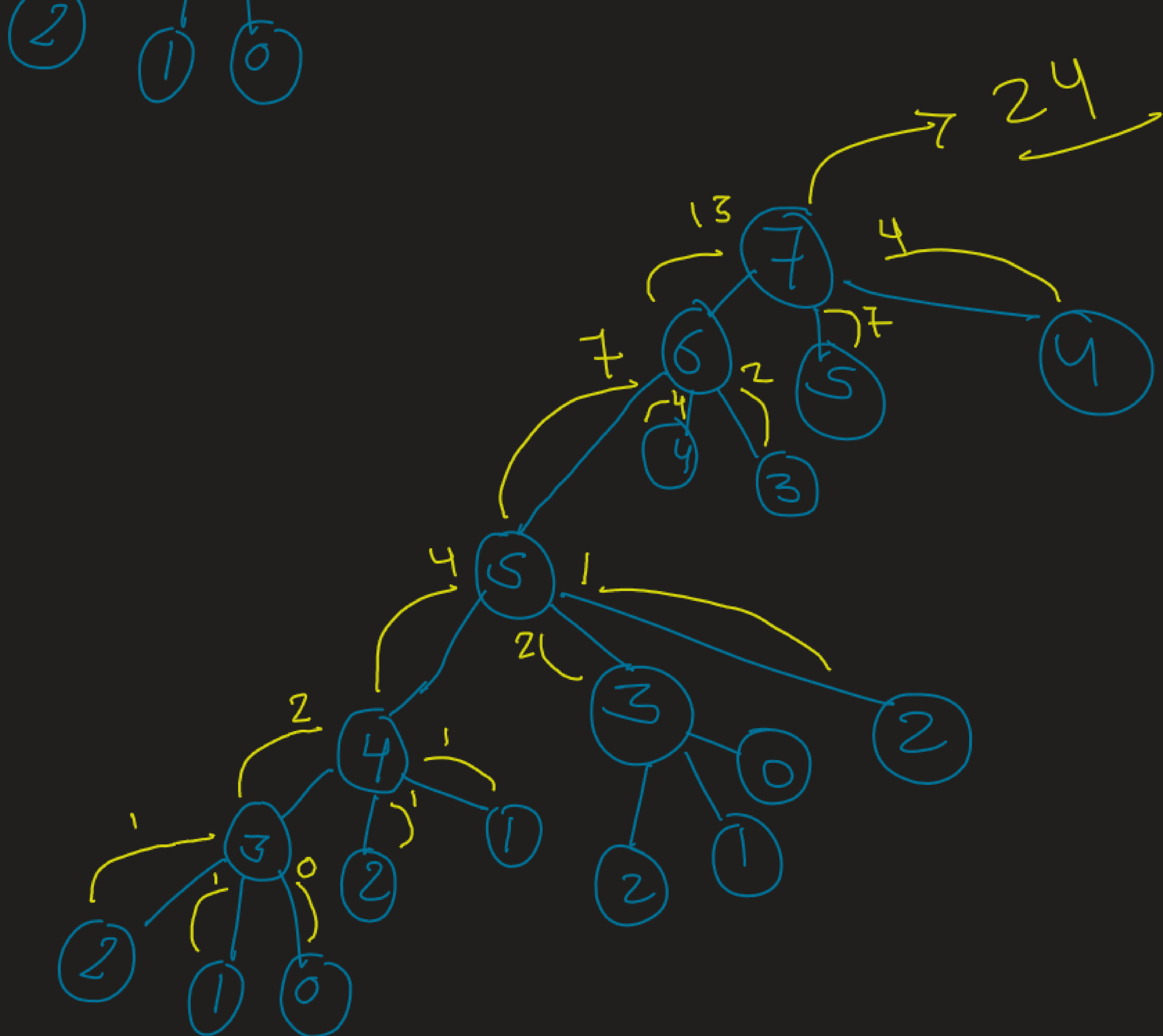
6^{th} –|– → 13

* Tree Diagram:-

$n = 7$



→ solving small problems (subproblems) to solve a Big Problem.



24
13
7
4
6
5
4
7
4
2
4
3
5
4
1
2
4
3
2
2
1
3
2
1
0
1
1
2
1
2
1
0
2
1
0

: Why It's taking Exponential time?

→ Overlapping Sub problems (Repitative work)

→ Means when n is increases, the amount of sub-problem increases

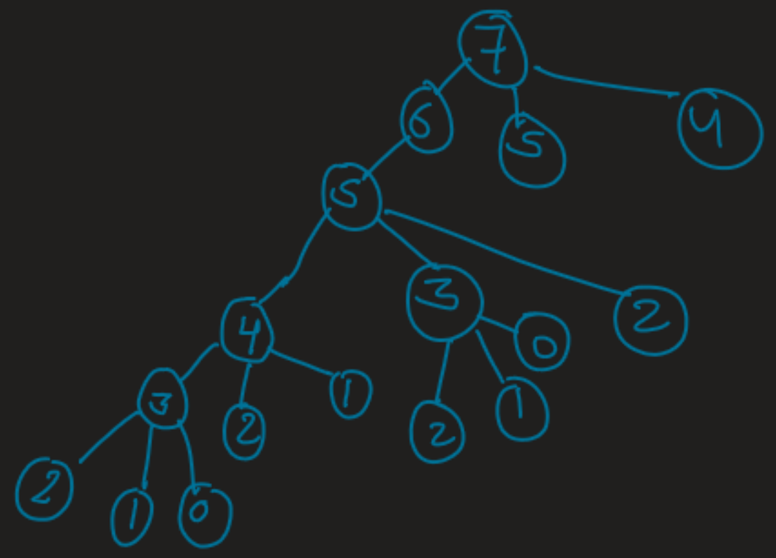
→ Solving this repetative problems again and again is increasing time.

→ This concludes that we could optime time in our recursion.

→ Now this hints to DP.

→ So for recursion, we apply Top-Down DP, so lets do Memoization.

* let's identify Base cases:-



→ if (n == 1) return 1
→ if (n == 2) return 1
→ if (n == 0) return 0

* For rest part:-

(we just need previous 3 values):

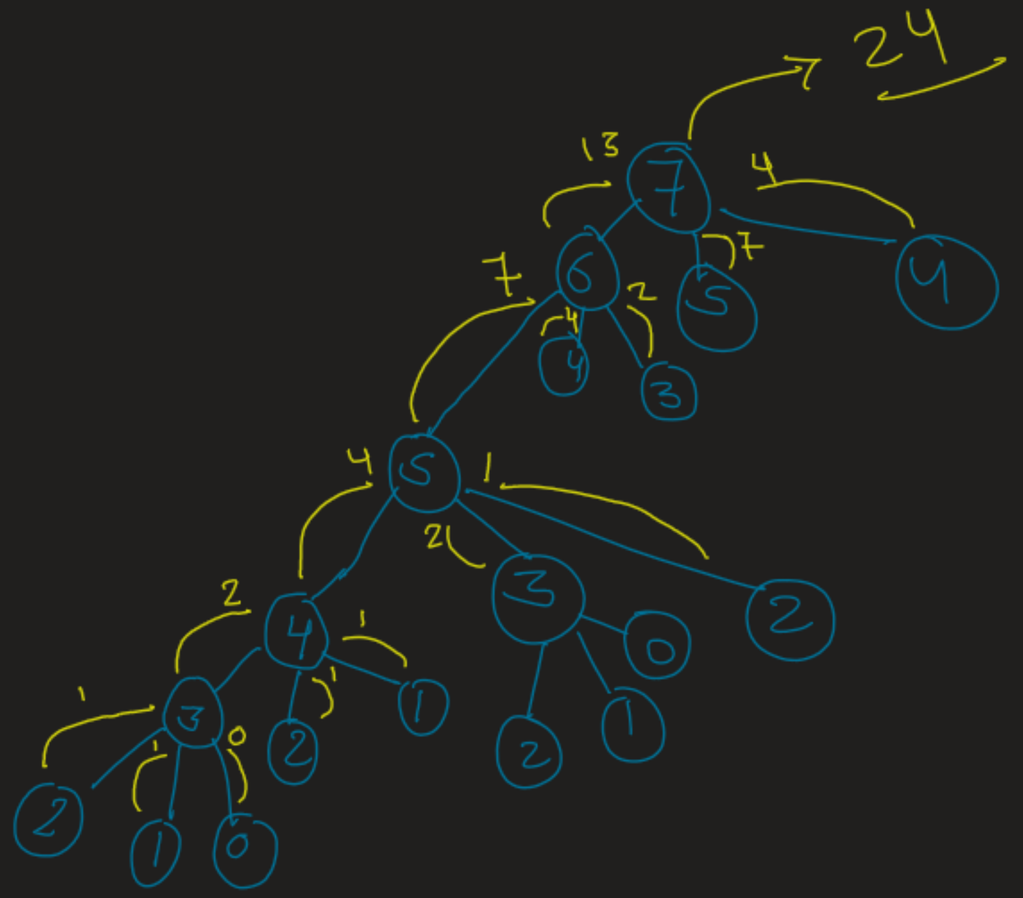
→ int prevVal1 (find recursively)
int prevVal2 (find recursively)
int prevVal3 (find recursively)

→ return sum of all values

⇒ return sum of all

~ Recursive code:-

```
int nthTribb(int n){
    if(n==0) return 0;
    if(n==1) return 1;
    if(n==2) return 1;
    int prevVal1 = nthTribb(n-1);
    int prevVal2 = nthTribb(n-2);
    int prevVal3 = nthTribb(n-3);
    return prevVal1 + prevVal2 + prevVal3;
}
```



⇒ Now memoize it (Apply DP)

⇒ let's code it